DuocUC® | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Documentación Técnica del Sistema "Ticki-TI"

Contenido

Introducción 3

Desarrollo del Notebook (Entrenamiento del Modelo) 3

Arquitectura del Sistema (Desarrollo en Streamlit y Backend) 5

1. Archivo: crear_usuarios.py 5

2. Archivo: gestionar_usuarios.py 5

3. Archivo: database.py (Módulo de Persistencia) 6

4. Archivo: app.py (Interfaz Gráfica y Orquestador) 6

Futuras Mejoras (Roadmap de Escalabilidad y Pase a Producción) 8

Introducción

El presente documento constituye la Documentación Técnica Oficial del Producto Mínimo Viable (MVP) para el sistema Ticki-TI. Su propósito es proveer a desarrolladores, arquitectos de *software* y equipos de operaciones de TI una visión profunda de la estructura interna, el código fuente y la arquitectura de datos que sustentan la plataforma. A lo largo de este manual se detallan los procedimientos de ingeniería de datos y el entrenamiento de los algoritmos de Inteligencia Artificial (ML), así como la orquestación del *backend* y la interfaz web construida en *Streamlit*. Finalmente, se establece la hoja de ruta técnica (Roadmap) con las optimizaciones arquitectónicas necesarias para el escalamiento y despliegue del sistema en un entorno de producción de alta disponibilidad.

Desarrollo del Notebook (Entrenamiento del Modelo)

La siguiente sección documenta la estructura y el flujo de ejecución del cuaderno de Jupyter utilizado para desarrollar el núcleo predictivo de Inteligencia Artificial del sistema Ticki-TI. Este entorno orquesta el ciclo de vida completo de los datos, abarcando desde la ingesta y limpieza del historial de tickets, hasta la vectorización de texto, el entrenamiento de los algoritmos de clasificación (Área y Prioridad) y la exportación de los artefactos finales listos para su integración en el *backend*. A continuación, se detalla de forma secuencial el propósito técnico y funcional de cada celda ejecutada:

- **Celda 0 (Título y configuración):** Instala e importa las librerías esenciales (Pandas, Scikit-learn, Matplotlib, Seaborn) y configura el entorno para visualizar los datos sin advertencias.
- **Celda 1 (Comprensión del Negocio):** Imprime los objetivos del proyecto (clasificar Área y Prioridad), explica la estrategia de simplificar la variable "Prioridad" agrupándola en dos grandes niveles (Estándar y Urgente) y define el umbral de éxito (F1-score >= 0.65).
- **Celda 2 (Carga de Datos):** Lee los cinco archivos Excel históricos (uno por cada área), estandariza los nombres de las columnas, les asigna su respectiva etiqueta de "Área" y los concatena en un único DataFrame principal con más de 5,700 registros.

- **Celda 3 (Limpieza de Texto):** Define y aplica una función basada en expresiones regulares (re) para limpiar el texto (Asunto + Descripción). Esta función convierte todo a minúsculas y elimina URLs, correos electrónicos, direcciones IP y caracteres especiales. Finalmente, descarta los tickets que tras la limpieza quedan con menos de 25 caracteres.
- **Celda 4 (Fusión Binaria y Encoders):** Agrupa las prioridades originales en las categorías "Estándar" y "Urgente". Luego, utiliza LabelEncoder para transformar las etiquetas de texto de "Área" y "Prioridad" en números que los algoritmos puedan procesar, definiendo finalmente las variables predictoras (X) y objetivo (y).
- **Celda 5 (División Train/Test):** Divide el conjunto de datos limpio en dos partes: 80% para entrenar el modelo y 20% para probarlo, asegurando que la proporción de las diferentes áreas se mantenga constante en ambos grupos (stratify).
- **Celda 6 (Entrenamiento de Modelos):** Descarga una lista de "palabras vacías" (*stopwords*) en español y le agrega palabras de ruido corporativo (nombres, saludos). Construye dos *pipelines*: uno usando el algoritmo LinearSVC para predecir el Área, y otro usando LogisticRegression para predecir la Prioridad. Ambos pipelines vectorizan el texto (TF-IDF), filtran el ruido y entrenan los modelos matemáticos.
- **Celda 7 (Evaluación Numérica):** Pone a prueba los modelos recién entrenados utilizando el 20% de datos reservados. Genera e imprime los reportes de clasificación que muestran la precisión, exhaustividad (*recall*) y el *F1-score*, validando que ambos algoritmos superan el umbral de aceptación del 65%.
- **Celda 8 (Evaluación Visual):** Genera e imprime las Matrices de Confusión (mapas de calor) usando Matplotlib y Seaborn. Esto permite ver visualmente dónde aciertan los modelos y en qué categorías específicas se están confundiendo.
- **Celda 8.1 (Análisis de Matrices):** (Celda de texto) Explica los resultados de los gráficos anteriores, señalando que el modelo de áreas acierta mucho pero se confunde levemente entre monitoreo e infraestructura, y que el modelo de prioridad prefiere sobrestimar la urgencia (falsos positivos), lo cual es favorable en un contexto de soporte técnico.

- **Celda 8.2 (Importancia de Variables):** Extrae de las matemáticas de los modelos cuáles fueron las palabras clave que más influyeron en las decisiones. Genera gráficos de barras que muestran el "top 10" de palabras que el algoritmo asocia con cada Área específica, y otro gráfico que muestra qué palabras empujan a clasificar un ticket como "Estándar" o "Urgente".
- **Celda 8.3 (Análisis de Variables):** (Celda de texto) Concluye que el modelo logra aprender el vocabulario técnico de cada equipo. Además, detecta que una palabra clave resultó ser el nombre de un cliente ("agunsa"), lo que alerta sobre la necesidad de aplicar una anonimización de datos (Ley Nº 21.719) antes de pasar a producción.
- **Celda 9 (Exportación):** Utiliza la librería joblib para "empaquetar" y guardar físicamente los dos modelos entrenados y sus decodificadores en archivos (.joblib). Finalmente, define la función maestra predecir_nuevo_ticket, que será el motor a utilizar en el backend para recibir texto, limpiarlo y devolver la sugerencia de la Inteligencia Artificial.

Arquitectura del Sistema (Desarrollo en Streamlit y Backend)

1. Archivo: crear_usuarios.py

Explicación: Este script actúa como el inicializador del sistema. Al ejecutarse, llama a la función de configuración de la base de datos (creando las tablas si no existen según el archivo SQL) y luego inyecta un "usuario semilla" o administrador inicial (Carlos Mendoza) con sus credenciales. Esto asegura que el sistema no esté vacío en el primer despliegue y que el equipo tenga una cuenta válida para probar el inicio de sesión.

2. Archivo: gestionar_usuarios.py

Explicación: Es un script de administración o mantenimiento. Dado que el MVP no tiene un panel de control avanzado en la web para gestionar a los empleados, este archivo permite a un desarrollador o administrador de base de datos modificar información directamente desde la terminal. Descomentando líneas específicas de código, se puede actualizar el nombre, cambiar la contraseña encriptada o desactivar (baja lógica o *soft delete*) a un técnico sin borrar su historial de tickets.

3. Archivo: database.py (Módulo de Persistencia)

Explicación por bloques:

- **Conexión y Setup:** Utiliza sqlite3 para crear un túnel seguro (usando *context managers* con yield) hacia la base de datos local ticki_ti.db. Aquí se configuran restricciones como las llaves foráneas (foreign_keys = ON) para asegurar la integridad referencial.
- **Gestión de Credenciales:** Contiene las funciones para registrar y validar el login. Utiliza la librería de ciberseguridad bcrypt para encriptar las contraseñas con un *salt* aleatorio, garantizando que ni siquiera un administrador pueda leer la clave real en la base de datos.
- **Operaciones CRUD:** Funciones auxiliares para listar, modificar nombres y desactivar técnicos de manera lógica (activo = 0).
- **Inyección Transaccional (Guardar Ticket):** La función más crítica. Inserta el nuevo ticket, asegurando que se escriba simultáneamente en tres tablas distintas (ticket_requerimiento, procesamiento_ia, validacion_humana). Compara la sugerencia de la máquina con la decisión final del humano y guarda un "1" si el técnico aceptó el pronóstico de la IA o un "0" si lo modificó, vital para futuras auditorías.

4. Archivo: app.py (Interfaz Gráfica y Orquestador)

Explicación por bloques:

- **Configuración y Carga en Caché (Bloque 1 y 2):** Inicia la aplicación web usando Streamlit. Carga en memoria caché (@st.cache_resource) el motor de visión artificial (EasyOCR) y los modelos matemáticos (.joblib) una sola vez al arrancar, para que la interfaz sea rápida y no tenga que cargar los archivos pesados cada vez que un técnico sube un ticket. También define la función de limpieza y anonimización de texto.

- **Control de Estado (Bloque 3):** Configura variables persistentes (st.session_state) para recordar quién inició sesión, qué resultados dio la IA y cuántos tickets se han acumulado, evitando que la página web "olvide" los datos si el usuario hace clic en otro botón.
- **Módulo de Autenticación (Vista 1):** Muestra el formulario de inicio de sesión. Si el correo y la contraseña coinciden con la base de datos, otorga acceso a la vista principal.
- **Panel de Procesamiento y Flujo IA (Vista 2):** Es el orquestador principal. Permite al técnico elegir si escribirá el ticket a mano o subirá una imagen. Al presionar "Procesar", el código enruta la imagen hacia el OCR (si aplica), limpia el texto para proteger datos sensibles y lo inyecta a la Inteligencia Artificial para obtener las sugerencias de Área y Prioridad.
- **Validación Humana y Exportación:** Tras la predicción de la IA, detiene el flujo para que el técnico valide (*Human-in-the-loop*). Una vez confirmado, guarda la transacción en la base de datos y agrega el registro a una tabla temporal (Pandas DataFrame) que permite descargar un reporte en formato Excel (.xlsx) consolidando todo lo trabajado en la sesión.

Futuras Mejoras (Roadmap de Escalabilidad y Pase a Producción)

Si bien la arquitectura del MVP actual resuelve exitosamente la clasificación automatizada y el esquema *Human-in-the-Loop*, la transición a un entorno de producción de alta concurrencia requerirá implementaciones estructurales clave. A nivel de persistencia de datos, se planifica la migración de la base de datos relacional local (SQLite) hacia un clúster NoSQL gestionado en la nube (MongoDB Atlas), optimizando el manejo de cargas JSON asíncronas. En la capa de *Frontend* (Streamlit), se sustituirá el renderizado estático del reloj por un componente asíncrono en tiempo real, junto con mejoras integrales de CSS para evitar conflictos visuales en modos de sistema dispares. A nivel algorítmico, el *pipeline* de *Machine Learning* será sometido a un reentrenamiento de sintonización fina (*fine-tuning*) incorporando técnicas de *Data Augmentation* (inyección de ruido tipográfico) para fortalecer la resiliencia del modelo ante capturas OCR de baja calidad. Simultáneamente, se integrará un modelo de Reconocimiento de Entidades Nombradas (NER) o expresiones regulares complejas para robustecer la anonimización de datos (Ley Nº 21.719) en el *backend*. Finalmente, el código fuente será empaquetado (Dockerización) y estabilizado mediante la optimización de cachés (@st.cache_resource) y la resolución de *bugs* de latencia detectados en las métricas de rendimiento, garantizando así un despliegue íntegro y mantenible.